KHora: A Model Checker for Knowing-How Logics

Rens Doodeman, Ziqi Wang, and Zixuan Chen

Wednesday 27th May, 2026

Abstract

We present KHora, a model checking tool for both the family of knowing-how logics.

Contents

1 Introduction

Knowing-how logics study an agent's ability to achieve goals by suitable action. This makes them particularly relevant to artificial intelligence, especially to planning, where the main question is whether a desired objective can be guaranteed by some plan.

The simplest knowing-how logic, $\mathcal{L}_{Kh}$, was first introduced in [3]. This logic captures an agent's ability to guarantee the achievement of a goal under a given condition. In this sense, the notion is somewhat idealized: even if an agent manages to achieve something under a condition, this does not necessarily mean that the agent genuinely knows how to do it. For example, winning a lottery does not imply knowing how to win it. In such cases, uncertainty becomes essential.

To address this, one may move to the uncertainty-based knowing-how logic $\mathcal{L}^U_{Kh}$ [1], which supports multi-agent reasoning. In this logic, an agent may have several indistinguishable plans for achieving the same purpose. This adds an epistemic feature to the basic knowing-how logic: plans are grouped into equivalence classes representing the agent's uncertainty about which plan is being followed. Finally, the uncertainty-based logic with regularity constraints reg-$\mathcal{L}^U_{Kh}$ [2] connects this approach with formal language theory by representing such equivalence classes using finite automata.

In this work, we present KHora, a model-checking tool for the family of knowing-how logics, based on the algorithms in [2].

The repository is available on GitHub at:
`https://github.com/Knowing-How-Logics/haskell-knowing-how-logics`.

2 Basic Knowing How Logic $\mathcal{L}_{Kh}$

In this section we model the language of $\mathcal{L}_{Kh}$ introduced in [3]. This captures the basic idea of an agent knowing how to achieve a goal under certain condition. We implement an explicit model-checker for this logic with a bounded plan depth, along with `QuickCheck` generators for random testing of the semantics.

2.1 Preliminaries

Definition 2.1. (Syntax) Given a set of proposition letters $Prop$, the language $\mathcal{L}_{Kh}$ is defined by

$$\varphi ::= \top \mid p \mid \neg\varphi \mid \varphi \wedge \varphi \mid Kh(\varphi, \varphi),$$

where $p \in Prop$.

Definition 2.2 (LTS)**.** A *labelled transition system* (LTS) is a tuple $\mathcal{M} = (S, (R_a)_{a\in Act}, V)$, where:

- S is a non-empty set of *states*,
- $(R_a)_{a\in Act}$ is a family of *binary relations* on S, each labelled by an *action* $a \in Act$,
- $V : S \to 2^{Prop}$ is a *valuation function.*

We write $s \xrightarrow{a} t$ if $(s,t) \in R_a$. For a plan $\sigma = a_1 \dots a_n \in Act^*$, we write $s \xrightarrow{\sigma} t$ if there exist $s_1, \dots, s_{n-1}$ such that $s \xrightarrow{a_1} s_1 \xrightarrow{a_2} \cdots \xrightarrow{a_n} t$. A plan $\sigma = a_1 \dots a_n$ is *strongly executable* at s iff for every $0 \leq k < n$ and every t such that $s \xrightarrow{\sigma_k} t$, the state t has at least one a_{k+1}-successor.

Definition 2.3 (Semantics)**.** Let $\mathcal{M}$ be an LTS, and s be a state.

$\mathcal{M}, s \models \top$	*iff*	*always*
$\mathcal{M}, s \models p$	*iff*	$p \in V(s)$
$\mathcal{M}, s \models \neg\varphi$	*iff*	$\mathcal{M}, s \not\models \varphi$
$\mathcal{M}, s \models \varphi \wedge \psi$	*iff*	$\mathcal{M}, s \models \varphi$ and $\mathcal{M}, s \models \psi$
$\mathcal{M}, s \models Kh(\psi, \varphi)$	*iff*	there exists a $\sigma \in Act^*$ such that for all s' such that $\mathcal{M}, s' \models \psi$: σ is strongly executable at s' and for all t such that $s' \xrightarrow{\sigma} t$, $\mathcal{M}, t \models \varphi$

The $Kh(\psi, \varphi)$ can be interpreted as "the agent knows how to achieve φ given ψ". Notice that the Kh is a global modality. Namely, either it holds for all states, or none of them.

2.2 Haskell Representation

```
data Form = P Proposition | Neg Form | Conj Form Form | KH Form Form | T
    deriving (Eq, Show, Ord)
```

Following Definition 2.2, we model the LTS as follows. When creating a LTS, consider that states is a non-empty list and therefore must use the :| constructor, i.e. (n :| [n+1..]).

```
data AbilityMap = LTS {
    states :: NonEmpty State,
    transitions :: Relations,
    valuation :: Valuation
} deriving(Show)
```

```
data SEFlag = SEOk | SEBad
    deriving (Eq, Show, Ord)

type SEComponent = (State, ([State], SEFlag))
-- The first State records the original precondition state.
-- The list records the current set of states reachable after the current action prefix.

type BadPairComponent = ((State, State), [State])
-- ((t1,t2), xs) tracks states currently reachable from t1.
-- t2 is a bad target, i.e. a state not satisfying the goal formula.

data KHProductState = KHProductState
    { seComponentsKH  :: [SEComponent]
    , badComponentsKH :: [BadPairComponent]
    } deriving (Eq, Show, Ord)

normaliseStates :: [State] -> [State]
normaliseStates = sort . nub

normaliseSEComponent :: SEComponent -> SEComponent
normaliseSEComponent (s, (xs, flag)) =
    (s, (normaliseStates xs, flag))

normaliseBadComponent :: BadPairComponent -> BadPairComponent
normaliseBadComponent (pair, xs) =
    (pair, normaliseStates xs)

normaliseKHProductState :: KHProductState -> KHProductState
normaliseKHProductState st =
    KHProductState
```

```
        { seComponentsKH  = map normaliseSEComponent (seComponentsKH st)
        , badComponentsKH = map normaliseBadComponent (badComponentsKH st)
        }

allHaveSuccessor :: Relations -> [State] -> Action -> Bool
allHaveSuccessor rs xs a =
    all (hasSuccessor rs a) xs

hasSuccessor :: Relations -> Action -> State -> Bool
hasSuccessor rs a x =
    not (null (image (rA rs a) x))

stepSEComponent :: Relations -> Action -> SEComponent -> SEComponent
stepSEComponent rs a (s0, (xs, flag)) =
    case flag of
        SEBad ->
            (s0, (xs, SEBad))
        SEOk ->
            let okNext = allHaveSuccessor rs xs a
                xs'    = stepSet rs xs a
            in if okNext
               then (s0, (xs', SEOk))
               else (s0, (xs', SEBad))

stepBadComponent :: Relations -> Action -> BadPairComponent -> BadPairComponent
stepBadComponent rs a (pair, xs) =
    (pair, stepSet rs xs a)

initialKHProductState :: [State] -> [State] -> KHProductState
initialKHProductState phiStates negPsiStates =
    normaliseKHProductState $
        KHProductState
            { seComponentsKH =
                [ (s, ([s], SEOk))
                | s <- phiStates
                ]
            , badComponentsKH =
                [ ((t1, t2), [t1])
                | t1 <- phiStates
                , t2 <- negPsiStates
                ]
            }

acceptingKHProductState :: KHProductState -> Bool
acceptingKHProductState st =
    all seAccepts (seComponentsKH st)
    && all badPairAccepts (badComponentsKH st)
  where
    seAccepts :: SEComponent -> Bool
    seAccepts (_, (_, flag)) =
        flag == SEOk

    badPairAccepts :: BadPairComponent -> Bool
    badPairAccepts ((_, badTarget), xs) =
        badTarget `notElem` xs

stepKHProductState :: Relations -> Action -> KHProductState -> KHProductState
stepKHProductState rs a st =
    normaliseKHProductState $
        KHProductState
            { seComponentsKH =
                map (stepSEComponent rs a) (seComponentsKH st)
            , badComponentsKH =
                map (stepBadComponent rs a) (badComponentsKH st)
            }

findWitnessPSpaceBySets :: AbilityMap -> [State] -> [State] -> Maybe Plan
findWitnessPSpaceBySets m phiStates psiStates =
    pathTo acceptingKHProductState next initialState
  where
    rs :: Relations
    rs =
        transitions m
```

```
    acts :: [Action]
    acts =
        actionsOf rs

    allStates :: [State]
    allStates =
        toList (states m)

    negPsiStates :: [State]
    negPsiStates =
        [ s | s <- allStates, s `notElem` psiStates ]

    initialState :: KHProductState
    initialState =
        initialKHProductState phiStates negPsiStates

    next :: KHProductState -> [(Action, KHProductState)]
    next current =
        [ (a, stepKHProductState rs a current)
        | a <- acts
        ]

khHoldsByProductSearch :: AbilityMap -> [State] -> [State] -> Bool
khHoldsByProductSearch m phiStates psiStates =
    case findWitnessPSpaceBySets m phiStates psiStates of
        Just _ ->
            True
        Nothing ->
            False

truthSet :: AbilityMap -> Form -> [State]
truthSet m f =
    [ s | s <- toList (states m), isTrue (m, s) f ]

isTrue :: (AbilityMap, State) -> Form -> Bool
isTrue _ T =
    True
isTrue (m, s) (P p) =
    p `elem` valuationAt (valuation m) s
isTrue (m, s) (Neg f) =
    not (isTrue (m, s) f)
isTrue (m, s) (Conj f g) =
    isTrue (m, s) f && isTrue (m, s) g
isTrue (m, _) (KH f g) =
    khHoldsByProductSearch m phiStates psiStates
  where
    phiStates =
        truthSet m f

    psiStates =
        truthSet m g

-- Infix alias for the complete satisfaction relation
(|=) :: (AbilityMap, State) -> Form -> Bool
(|=) = isTrue

findWitness :: AbilityMap -> Form -> Form -> Maybe Plan
findWitness m f g =
    findWitnessPSpaceBySets m phiStates psiStates
  where
    phiStates =
        truthSet m f

    psiStates =
        truthSet m g
```

2.3 Parsing for $\mathcal{L}_{Kh}$

Formulas may be created in `ghci` using `parseForm`. We omit most of the code here. The following inputs are accepted:

- `p` or `P` followed by an integer n returns `P n`
- `T` returns `T`
- `!p` returns `Neg p`
- `p & q` returns `Conj p q`
- `KH p q` returns `KH p q`
- `p -> q` returns `Neg (Conj p (Neg q))` (as an abbreviation)
- `p v q` or `p V q` returns `Neg (Conj (Neg p) (Neg q))` (as an abbreviation)

```haskell
parseForm :: String -> Either ParseError Form
parseForm = parse (pForm <* eof) "input"

evalForm :: (AbilityMap, State) -> String -> Bool
evalForm (m, s) str =
    case parseForm str of
        Right f ->
            isTrue (m, s) f
        Left _ ->
            error "Invalid formula"
```

3 Uncertainty-based Knowing-how Logic with Regularity Constraints $reg\text{-}\mathcal{L}_{Kh}^{U}$

In the framework of *basic knowing how* we introduced above, an agent possesses the ability to achieve the goal φ given ψ if and only if he has a plan that is fail-proof, meaning that each partial execution must be completable. In scenarios where the agent lacks this ability, it is only because a sequence of actions cannot be generated due to certain environmental constraints. However, another scenario may also occur: the agent does not know which plan is adequate for the situation. All he can do is blindly apply a plan he thought might work, which may or may not be successful. Such *indistinguishability* among plans establishes an epistemic relation of *knowing that*.

We now introduce a new logic of *knowing how* for a multi-agent setting defined in [2], extending the LTS with a notion of epistemic indistinguishability between plans. The uncertainty of an agent is encoded by equivalence classes of plans, and each such class is represented by a finite automaton. More precisely, each equivalence class of indistinguishable plans is represented by the language recognised by a finite automaton.

In Theorem 2 from [2], it is shown that the model checking problem for this logic is in PTIME. The proof relies on reducing both conditions of the Kh_i-modality to graph reachability checks. In this section, we provide a concrete implementation of this algorithm in Haskell, closely following the constructions from the paper.

3.1 Preliminaries

Definition 3.1 (Syntax)**.** Given a set of proposition letters $Prop$ and a set of agents Agt, where $p \in Prop$ and $i \in Agt$, the language $reg\text{-}\mathcal{L}^{U}_{Kh}$ is defined by

$$\varphi := p \mid \neg\varphi \mid \varphi \vee \varphi \mid Kh_i(\psi, \varphi).$$

We first introduce the automata used to represent classes of plans.

Definition 3.2 (Automaton)**.** A non-deterministic finite automaton is a tuple $\mathcal{A} = (Q, Act, \delta, I, F)$, where:

- Q is a finite set of states,
- Act is a finite alphabet of actions,
- $\delta : Q \times Act \to 2^Q$ is a transition function,
- $I \subseteq Q$ is the set of initial states,
- $F \subseteq Q$ is the set of accepting (final) states.

To connect an automaton with the underlying transition system, we use the following product digraph.

Definition 3.3 (Digraph)**.** Given a reg-LTS^U $\mathcal{S}$ and an automaton $\mathcal{A} = (Q, Act, \delta, I, F)$, we define a digraph $G = (V, E)$ as follows:

- $V = Q \times S$, representing the combined states of the automaton and the underlying transition system;
- E is the set of edges such that $((q, t), (q', t')) \in E$ if and only if there exists an action $a \in Act$ such that:
  1. $q \xrightarrow{a} q'$ in $\mathcal{A}$;
  2. $(t, t') \in R_a$.

This construction synchronizes transitions of the automaton with transitions of the LTS under the same action, allowing us to track which runs of the automaton are realizable in the model.

We next recall the relational notions needed for the semantics.

Definition 3.4. For $\pi \subseteq Act^*$, $u \in S$, and $U \subseteq S$, define

$$R_\pi := \bigcup_{\sigma\in\pi} R_\sigma, \qquad R_\pi(u) := \bigcup_{\sigma\in\pi} R_\sigma(u), \qquad R_\pi(U) := \bigcup_{u\in U} R_\pi(u).$$

A set of plans $\pi \subseteq Act^*$ is *strongly executable* at $u \in S$ iff every plan $\sigma \in \pi$ is strongly executable at u. This gives rise to the definition $SE(\pi) := \bigcap_{\sigma\in\pi} SE(\sigma)$, which collects all states at which every plan in π is strongly executable.

These notions can be lifted from plan sets to automata via their accepted languages.

Definition 3.5. In terms of automata, $\mathrm{SE}(L(\mathcal{A})) := \bigcap_{\sigma \in L(\mathcal{A})} \mathrm{SE}(\sigma)$ collects all states at which every plan in $L(\mathcal{A})$ is strongly executable, and $R_{L(\mathcal{A})} := \bigcup_{\sigma \in L(\mathcal{A})} R_\sigma$ collects all transitions realisable by some plan in $L(\mathcal{A})$.

We now define reg-LTS^U models.

Definition 3.6. A reg-LTS^U is a tuple $\mathcal{S} = (S, (R_a)_{a \in Act}, (U_i)_{i \in Agt}, V)$, where the elements of U_i associated with an agent i are finite automata, namely,

$$\emptyset \neq U_i = \{\mathcal{A}_1, \mathcal{A}_2, \ldots\}.$$

Here, $L(\mathcal{A}_i)$ denotes the language accepted by $\mathcal{A}_i$. Moreover, for every $\mathcal{A}_m, \mathcal{A}_n \in U_i$ such that $m \neq n$, we have

$$L(\mathcal{A}_m) \cap L(\mathcal{A}_n) = \emptyset.$$

Finally, we can state the semantics of the language.

Definition 3.7 (Semantics)**.** Let $\mathcal{S} = (S, (R_a)_{a \in Act}, (U_i)_{i \in Agt}, V)$ be a finite reg-LTS^U, and $s \in S$. The satisfaction relation $\models$ is defined as:

$\mathcal{S}, s \Vdash p$	*iff*	$p \in V(s)$
$\mathcal{S}, s \Vdash \neg\varphi$	*iff*	$\mathcal{S}, s \nVdash \varphi$
$\mathcal{S}, s \Vdash \psi \vee \varphi$	*iff*	$\mathcal{S}, s \Vdash \psi$ or $\mathcal{S}, s \Vdash \varphi$
$\mathcal{S}, s \Vdash Kh_i(\psi, \varphi)$	*iff*	there is a finite automaton $\mathcal{A} \in U_i$, such that (1) $[\![\psi]\!]^{\mathcal{S}} \subseteq SE(L(\mathcal{A}))$ and (2) for every $t \in [\![\psi]\!]^{\mathcal{S}}$, $R_{L(\mathcal{A})}(t) \subseteq [\![\varphi]\!]^{\mathcal{S}}$,

where $[\![\psi]\!]^{\mathcal{S}} := \{s \in S \mid \mathcal{S}, s \Vdash \psi\}$ is the set of all states satisfying ψ.

The $Kh_i(\psi, \varphi)$ can be interpreted as "when ψ is the case, the agent i knows how to make φ true". Kh_i is also a global modality.

3.2 Haskell Representation

The syntax is modelled following Definition 3.1.

```
type Agent = Int

data RegForm = Prop Proposition | Not RegForm | Disj RegForm RegForm | KHI Agent RegForm RegForm
    deriving (Eq, Show, Ord)
```

The product digraph from Definition 3.3 is implemented below. The helper functions `getAutNext` and `getLtsNext` retrieve successor states in the automaton and LTS respectively. They are omitted here.

```
type GVertex = (AState, State) -- (Automaton state, LTS state)
type GEdge = (GVertex, GVertex)

data Digraph = Digraph
    { vSet :: [GVertex]
    , eSet :: [GEdge]
    } deriving (Eq, Show, Ord)
```

The following functions implement the relational notions from Definition 3.4.

```haskell
type PlanSet = [Plan]

-- R_pi(u) = union of R_sigma(u) for all sigma in pi
rPiU :: Relations -> State -> PlanSet -> [State]
```

```haskell
-- R_pi(X) = union of R_pi(u) for all u in X
rPiX :: Relations -> [State] -> PlanSet -> [State]
```

```haskell
-- SE(sigma) = set of all states at which sigma is strongly executable
seSigma :: [State] -> Relations -> Plan -> [State]
```

```haskell
-- SE(pi) = intersection of SE(sigma) for all sigma in pi
sePi :: [State] -> Relations -> PlanSet -> [State]
```

The functions above are not used directly in the model-checking algorithm, but are included to keep the implementation close to the relational notions from the paper. Definition 3.5 is not implemented separately, since it expresses the same ideas from the automata-language perspective.

Finally, the reg-LTS^U model implements Definition 3.6.

```haskell
type Uncertainty = [(Agent, [Automaton])] -- U_i = {A_1,...}, for each agent i

data RegLTSU = RegLTSU
    { statesM     :: [State]
    , relationsM  :: Relations
    , uncertainty :: Uncertainty
    , valuationM  :: Valuation -- shared valuation type from the LTS module
    } deriving (Eq, Show, Ord)

-- Given an agent i, return U_i = [A_1,...]
getAgentAuts :: RegLTSU -> Agent -> [Automaton]
getAgentAuts m agent =
    fromMaybe [] (lookup agent (uncertainty m))
```

3.3 Model checker in Haskell

We now discuss the implementation model checking algorithms for $reg\text{-}\mathcal{L}^U_{Kh}$ in Haskell.

Given a finite reg-LTS^U $\mathcal{S} = (S, (R_a)_{a\in Act}, (U_i)_{i\in Agt}, V)$, a state $s \in S$, and a formula $Kh_i(\varphi, \psi)$, we check whether $\mathcal{S}, s \models Kh_i(\varphi, \psi)$ by iterating over all automata $\mathcal{A} \in U_a$ and verifying two conditions:

1. $[\![\varphi]\!]^{\mathcal{S}} \subseteq \mathrm{SE}(L(\mathcal{A}))$
2. $R_{L(\mathcal{A})}([\![\varphi]\!]^{\mathcal{S}}) \subseteq [\![\psi]\!]^{\mathcal{S}}$

Once we find such automaton, the formula $Kh_i(\varphi, \psi)$ is then satisfied. Since $L(\mathcal{A})$ may be infinite, directly enumerating all plans is not feasible. Following [2], we handle each condition via a separate algorithm, both of which reduce the problem to reachability checks on finite graphs.

To perform the reachability checks in PTIME, we use the following *depth-first search* algorithm:

Require: Digraph $G = (V, E)$, start vertex v_0, target set $T_0 \subseteq V$
1: DFS($G, v_0, T_0, \emptyset$)

function DFS($G, v, T, visited$)
  if $v \in T$ **then return true**
  end if
  if $v \in visited$ **then return false**
  end if
  $visited \leftarrow visited \cup \{v\}$
  for $w \in \text{successors}(G, v)$ **do**
    if DFS($G, w, T, visited$) **then return true**
    end if
  end for
  return false
end function

The algorithm above runs in $O(|V| + |E|)$, which is polynomial.
The algorithm that checks Condition (1) from [2] works as follows:

Require: reg-LTS^U $\mathcal{S}$, state $s \in S$, automaton $\mathcal{A} = (Q, Act, \delta, I, F)$
function CheckSE-Original($\mathcal{S}, s, \mathcal{A}$)
  Build the product digraph $G = (Q \times S,\ E)$
  for $a \in Act$ **do**
    for $t \in S$ such that $R_a(t) = \emptyset$ **do**
      for $q \in Q$ such that $\delta(q, a) \neq \emptyset$ **do**
        for $q_0 \in I$ **do**
          if (q, t) is reachable from (q_0, s) in G **then**
            return true
          end if
        end for
      end for
    end for
  end for
  return false
end function

Instead of using the nested for-loops, we first collect all *bad vertices* via set comprehension, i.e.

$$Bad = \{(q, t) \in Q \times S \mid \exists\, a \in Act : \delta(q, a) \neq \emptyset \text{ and } R_a(t) = \emptyset\},$$

and then perform a single DFS reachability check from each initial vertex (q_0, s) to the set Bad. This is equivalent to the original procedure, but avoids repeated reachability checks.

A bad vertex (q, t) represents a situation where the automaton expects to perform some action a at state q, but the LTS cannot execute a at state t. If such a vertex is reachable, it means that some plan in $L(\mathcal{A})$ fails to be strongly executable.

Remark: This function might be more accurately called `checkNotSE` rather than `checkSE`, since it returns true exactly when strong executability fails.

Require: reg-LTS^U $\mathcal{S} = (S, (R_a), (U_i), V)$, state $s \in S$, automaton $\mathcal{A} = (Q, Act, \delta, I, F)$
function CheckSE($\mathcal{S}, s, \mathcal{A}$)
  Build product digraph $G = (Q \times S,\ E)$
  $Bad \leftarrow \{(q, t) \in V \mid \exists\, a \in Act : \delta(q, a) \neq \emptyset \text{ and } R_a(t) = \emptyset\}$
  for $q_0 \in I$ **do**
    if DFS((q_0, s)) with target Bad **then return true**
    end if

7: **end for**
8: **return false**
9: **end function**

Require: reg-LTS^U $\mathcal{S}$, automaton $\mathcal{A}$, truth set $[\![\varphi]\!]^{\mathcal{S}}$
1: **function** CHECKCOND1($\mathcal{S}, \mathcal{A}, [\![\varphi]\!]^{\mathcal{S}}$)
2: **for** $s \in [\![\varphi]\!]^{\mathcal{S}}$ **do**
3: **if** CHECKSE($\mathcal{S}, s, \mathcal{A}$) **then return false**
4: **end if**
5: **end for**
6: **return true**
7: **end function**

In Haskell:

```haskell
-- Compute the set of bad vertices in G.
-- A vertex (q, t) is bad iff there exists an action a such that:
--   (i)  the automaton has a transition from q under a, and
--   (ii) the LTS has no successor from t under a.
badVertices :: RegLTSU -> Automaton -> [GVertex]
badVertices m aut =
    nub [ (q, t)
        | q <- autStates aut
        , t <- statesM m
        , a <- autAlphabet aut
        , not (null (getAutNext aut q a))  --  delta(q,a) != empty
        , null (getLtsNext m t a)          --  R_a(t) == empty
        ]

-- checker whether there is a path from (q0, s) to a bad vertex.
-- If so, return True. This means that s not in SE(L(A)).
-- Else, return False. This means that s in SE(L(A)).
checkSE :: RegLTSU -> State -> Automaton -> Bool
checkSE m s aut =
    let g      = buildDigraph m aut
        bad    = badVertices m aut
        starts = [(q0, s) | q0 <- autInitial aut]
    in existsReachableFromAny (`elem` bad) (successors g) starts

checkCond1 :: RegLTSU -> Automaton -> [State] -> Bool
checkCond1 m aut =
    all (\s -> not (checkSE m s aut))
```

Let $\mathcal{A}_1$ and $\mathcal{A}_2$ be two automata. To check whether $L(\mathcal{A}_1) \cap L(\mathcal{A}_2) \neq \emptyset$, it suffices to construct the product automaton and check whether any accepting state pair in $F_1 \times F_2$ is reachable from some initial state pair in $I_1 \times I_2$.

Require: Automata $\mathcal{A}_1 = (Q_1, Act_1, \delta_1, I_1, F_1)$ and $\mathcal{A}_2 = (Q_2, Act_2, \delta_2, I_2, F_2)$
1: **function** INTERSECTIONNONEMPTY($\mathcal{A}_1, \mathcal{A}_2$)
2: //Build product automaton
3: $V \leftarrow Q_1 \times Q_2$
4: $E \leftarrow \{((q_1, q_2), (q_1', q_2')) \mid a \in Act_1 \cap Act_2,\ q_1 \xrightarrow{a} q_1' \in \delta_1,\ q_2 \xrightarrow{a} q_2' \in \delta_2\}$
5: $G \leftarrow (V, E)$
6: $T \leftarrow F_1 \times F_2$
7: **for** $(q_1, q_2) \in I_1 \times I_2$ **do**
8: **if** DFS((q_1, q_2)) with target T **then return true**
9: **end if**
10: **end for**
11: **return false**
12: **end function**

To check Condition (2), it suffices to check whether

$$L(\mathcal{A}) \cap \bigcup \{ L(\mathcal{A}_{(t_1,t_2)}) \mid t_1 \in [\![\varphi]\!]^{\mathcal{S}},\, t_2 \in [\![\neg\psi]\!]^{\mathcal{S}} \} = \emptyset,$$

where $\mathcal{A}_{(t_1,t_2)} = (S, Act, \delta', \{t_1\}, \{t_2\})$ is the path automaton whose transitions mirror the LTS, i.e. $\delta'(t,a) = \{t' \mid (t,t') \in R_a\}$. Its language is precisely $L(\mathcal{A}_{(t_1,t_2)}) = \{\sigma \in Act^* \mid t_2 \in R_\sigma(t_1)\}$.

By distributivity of $\cap$ over $\bigcup$, this is equivalent to checking that

$$L(\mathcal{A}) \cap L(\mathcal{A}_{(t_1,t_2)}) = \emptyset \quad \text{for all } (t_1,t_2) \in [\![\varphi]\!]^{\mathcal{S}} \times [\![\neg\psi]\!]^{\mathcal{S}}.$$

Require: reg-LTS^U $\mathcal{S}$, automaton $\mathcal{A}$, truth sets $[\![\varphi]\!]^{\mathcal{S}}$ and $[\![\neg\psi]\!]^{\mathcal{S}}$
1: **function** CHECKCOND2($\mathcal{S}, \mathcal{A}, [\![\varphi]\!]^{\mathcal{S}}, [\![\neg\psi]\!]^{\mathcal{S}}$)
2: **for** $t_1 \in [\![\varphi]\!]^{\mathcal{S}}$ **do**
3: **for** $t_2 \in [\![\neg\psi]\!]^{\mathcal{S}}$ **do**
4: **if** INTERSECTIONNONEMPTY($\mathcal{A}, \mathcal{A}_{(t_1,t_2)}$) **then return false**
5: **end if**
6: **end for**
7: **end for**
8: **return true**
9: **end function**

In Haskell:

```haskell
-- Construct A_(t1, t2)
-- This automaton accepts all plans from t1 to t2
buildPathAutomaton :: RegLTSU -> State -> State -> Automaton
buildPathAutomaton m t1 t2 = Automaton
    { autStates      = statesM m
    , autAlphabet    = acts
    , autTransitions = [ ((s, a), getLtsNext m s a) | s <- statesM m, a <- acts ]
    , autInitial     = [t1]
    , autFinal       = [t2]
    }
  where
    -- Collect all actions that appear in the LTS relations
    acts = nub [a | (a, _) <- relationsM m]

-- Check for each A: for all (t1,t2): L(A) \cap L(A_(t1, t2)) = empty?
-- We check the emptiness of the intersection by checking the reachability of the product graph(automaton)
-- If empty, then cond 2 is SAT
checkCond2 :: RegLTSU -> Automaton -> [State] -> [State] -> Bool
checkCond2 m aut phiStates negPsiStates = not (any violates pairs)
  where
    pairs = [(t1, t2) | t1 <- phiStates, t2 <- negPsiStates]
    -- Check the intersection. Non-empty => cond 2 is UNSAT.
    violates (t1, t2) =
        let pathAut = buildPathAutomaton m t1 t2
        in  intersectionNonEmpty aut pathAut
```

Following the semantics defined in Definition 3.7 and putting things together, we complete the model checker for $reg\text{-}\mathcal{L}^U_{Kh}$.

```haskell
-- [[phi]]= set of states that phi holds
truthSet :: RegLTSU -> RegForm -> [State]
truthSet m f = [s | s <- statesM m, isTrueReg (m, s) f]

-- Satisfaction relation for the propositional fragment
isTrueReg :: (RegLTSU, State) -> RegForm -> Bool
isTrueReg (m, s) (Prop p) =
```

```
    p `elem` valuationAt (valuationM m) s
isTrueReg (m, s) (Not f) =
    not (isTrueReg (m, s) f)
isTrueReg (m, s) (Disj f g) =
    isTrueReg (m, s) f || isTrueReg (m, s) g

-- Kh_a(phi, psi) holds iff there exists A in U_a such that
-- (1) [[phi]] is subset of SE(L(A))
-- (2) R_{L(A)}([[phi]]) is subset of [[psi]]
isTrueReg (m, _) (KHI agent phi psi) =
    case findWitnessAutomaton m agent phi psi of
        Just _ ->
            True
        Nothing ->
            False

-- Infix alias for the satisfaction relation
(||=) :: (RegLTSU, State) -> RegForm -> Bool
(||=) = isTrueReg

findWitnessAutomaton :: RegLTSU -> Agent -> RegForm -> RegForm -> Maybe Automaton
findWitnessAutomaton m agent phi psi =
    firstGood (getAgentAuts m agent)
  where
    phiStates =
        truthSet m phi

    negPsiStates =
        truthSet m (Not psi)

    isGoodAutomaton aut =
        checkCond1 m aut phiStates
        && checkCond2 m aut phiStates negPsiStates

    firstGood [] =
        Nothing
    firstGood (aut:auts)
        | isGoodAutomaton aut =
            Just aut
        | otherwise =
            firstGood auts
```

3.4 Parsing for $reg\text{-}\mathcal{L}^{U}_{Kh}$

Formulas may be created in `ghci` using `parseRegForm`. We omit most of the code here. The following inputs are accepted.

- `p` or `P` followed by an integer n returns `Prop n`
- `!p` returns `Not p`
- `p v q` or `p V q` returns `Disj p q`
- `KHi p q` returns `KHI i p q`, where `i` is an agent index
- `p -> q` returns `Disj (Not p) q` (as an abbreviation)
- `p & q` returns `Not (Disj (Not p) (Not q))` (as an abbreviation)

```
parseRegForm :: String -> Either ParseError RegForm
parseRegForm = parse (pRegForm <* eof) "input"

evalRegForm :: (RegLTSU, State) -> String -> Bool
evalRegForm (m, s) str =
```

```
    case parseRegForm str of
        Right f -> isTrueReg (m, s) f
        Left _  -> error "Invalid formula"
```

4 Web Interface

In this section we introduce a web-application to demo our code. The web-app is consistent of a single page allowing users to do the following:

- Parse single or multi agent formulas to our Haskell syntax
- Generate random single agent formulas in our Haskell syntax
- Generate single or multi agent models in our Haskell syntax using size-parameters
- Generate random single agent models in our Haskell syntax

The app also provides instructions on how to model-check.

Providing a user-interface (UI) is an additional feature to our project. Therefore we have kept our implementation extremely simple. We use a simple lightweight RESTful framework called scotty, and plain HTML and CSS is used for layout and styling. We serve a single plain JavaScript script to connect the UI to our Haskell code. We omit most of the code here.

The web-app is currently not deployed to the web. However, it is possible to run the app by running `stack build` and `stack run`.

```
main :: IO ()
main = scotty 3000 $ do
  middleware $ staticPolicy (noDots >-> addBase "static")

  get "/" $ do
    file "static/index.html"
```

5 Conclusion

References

[1] C. Areces, R. Fervari, A. R. Saravia, and F. R. Velázquez-Quesada. Uncertainty-based semantics for multi-agent knowing how logics. In *Proceedings of the 18th Conference on Theoretical Aspects of Rationality and Knowledge (TARK 2021)*, volume 335 of *EPTCS*, pages 23–37, 2021.

[2] S. Demri and R. Fervari. Model-checking for ability-based logics with constrained plans. In *Proceedings of the 37th AAAI Conference on Artificial Intelligence (AAAI-23)*, volume 37, pages 6305–6312, 2023.

[3] Y. Wang. A logic of knowing how. In *Logical Reasoning and Interaction (LORI'15)*, volume 9394 of *Lecture Notes in Computer Science (LNCS)*, pages 392–405. Springer, 2015.